
DSP Studio

An Interactive Web-Based Digital Signal Processing Workstation

Md Mostakim | ID: 2305063

Samiul Basir Adry | ID: 2305071

CSE 220 — Digital Signal Processing

What We Built

7 Modules + 1 Studio

1. Amplitude & Gain
 2. Time-Domain Processing (Convolution, Echo, Delay)
 3. Spectral Detection (Pitch)
 4. Sampling & Quantization
 5. Frequency & Filtering
 6. Voice Morphing
 7. Spectral Portal
- ★ **Studio** — chain any of the above

Engine

Backend: Python, FastAPI, NumPy, SciPy, Librosa

Frontend: React, Vite

Idea: every module visualizes one classical DSP algorithm end-to-end — upload/record audio → backend computes the real transform → animated, math-labelled result.

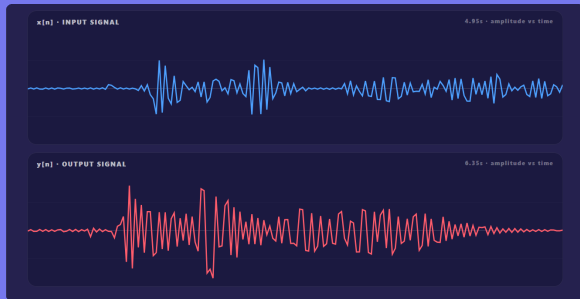
Amplitude & Gain

Algorithm

1. Read the input signal $x[n]$, one sample at a time.
2. Multiply that sample by a constant gain factor A .
3. Write the scaled value out as $y[n]$.

$$y[n] = A \cdot x[n]$$

The output (red) is an exact scaled copy of the input (blue) — same shape, taller peaks. Peak/RMS/clipping are reported so a gain past full scale is visible immediately.



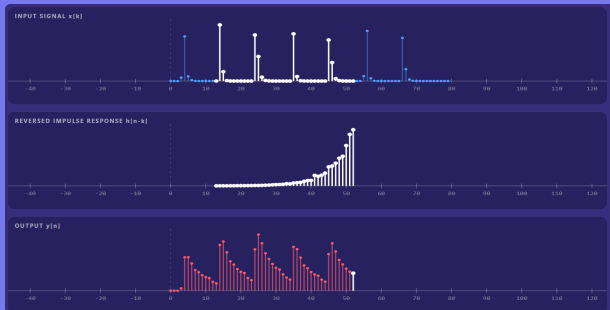
Input $x[n]$ vs. output $y[n]$, $A = 2$

Convolution

Algorithm

1. Take the input $x[n]$ and an impulse response $h[n]$ (e.g. a room's echo signature).
2. **Reverse** h in time.
3. **Slide** the reversed h across x , one sample shift at a time.
4. At each shift, **multiply** the overlapping samples and **sum** them — that sum is one output sample.
5. Repeat until the reversed h has slid completely past x .

$$y[n] = \sum_k x[k] h[n - k]$$



$x[k]$, reversed $h[n - k]$ sliding, and the accumulating output $y[n]$

Echo

Algorithm — feedback delay line

1. Keep the original (dry) signal.
2. Make a copy shifted later in time by D samples, and shrink its volume by a decay factor $\alpha < 1$.
3. Add that quieter, delayed copy back into the output.
4. **Repeat** steps 2–3 R times: each new repeat is delayed further and decays more than the last (feedback).
5. Sum every repeat with the original — several decaying, evenly-spaced echoes.

$$y[n] = x[n] + \sum_{r=1}^R \alpha^r x[n - rD]$$



Input $x[n]$ vs. output $y[n]$ — repeated, decaying reflections

Delay

Algorithm — single repeat

1. Keep the original (dry) signal.
2. Make **one** copy shifted later in time by D samples — no feedback, it is not fed back into itself.
3. Scale that single copy by a wet-mix factor α .
4. Add it to the original — exactly one distinct, un-decayed repetition.

$$y[n] = x[n] + \alpha \cdot x[n - D]$$

Compare to Echo: same building block ($x[n - D]$), used once instead of looped.



Input $x[n]$ vs. output $y[n]$ — one clean, delayed repeat

Spectral Detection — Pitch

Algorithm

1. Apply a window (Hann) to the signal, then take its **FFT** to get the spectrum $|X(f)|$.
2. Restrict the search to a chosen range $[f_{\min}, f_{\max}]$ — ignoring DC/rumble and inaudible highs.
3. Find the frequency bin with the **largest magnitude** in that range — the dominant frequency f_0 .
4. Map f_0 to the nearest musical note.
5. **Synthesize** a pure sine wave at f_0 so the detection can be checked by ear.



$$f_0 = \arg \max_{f \in [f_{\min}, f_{\max}]} |X(f)| \quad y[n] = A \sin(2\pi f_0 n / f_s)$$

A real signal spreads energy across a fundamental & overtones (left); the synthesized tone is one sharp spike at f_0 (right)

Sampling & Quantization

Quantization

1. Pick how many bits b to keep (e.g. 4-bit = 16 levels).
2. Compute the step size Δ spanning $[-1, 1]$.
3. **Round** each sample to the nearest allowed level, then clip.

$$y[n] = \Delta \cdot \text{round}(x[n]/\Delta), \quad \Delta = \frac{2}{2^b - 1}$$



Sample-and-Hold

1. Pick a hold factor N (simulates a lower sample rate).
2. Walk the signal in blocks of N samples.
3. **Hold** the block's first sample for the whole block (zero-order hold), instead of reading a fresh one.

$$y[n] = x(N \lfloor n/N \rfloor)$$

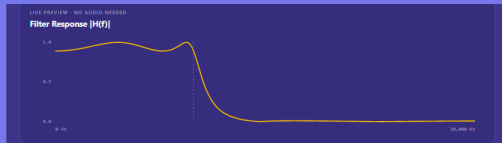


Frequency & Filtering

Algorithm

1. Choose a filter family, band type, cutoff(s) and order N .
2. **Design** the filter as SOS (second-order sections): a chain of small 2-pole/2-zero stages multiplied together, instead of one giant high-order formula — a high-order filter's raw coefficients can span a huge numeric range and go unstable; small cascaded stages stay well-behaved.
3. **Apply it twice** with `sosfiltfilt`: once forwards, once backwards. The two passes cancel each other's phase delay, so output stays time-aligned with input.

$$Y(f) = X(f) \cdot H(f)$$



The filter's own magnitude response $|H(f)|$ — this lowpass example passes $f \leq f_c$

Families & Bands

Butterworth • Chebyshev I/II • Elliptic • Bessel • Ideal
Lowpass • Highpass • Bandpass • Bandstop

Voice Morphing — Phase Vocoder

Algorithm (Pitch / Time-Stretch)

1. **STFT**: split into overlapping frames; get magnitude & phase of each, independently.
2. **Time-map**: to stretch by rate r , read analysis frames at $r \times$ the normal spacing (often between two real frames).
3. **Magnitude interpolation**: blend the two nearest frames to estimate that in-between spectrum.
4. **Phase updating**: estimate each bin's true frequency from the phase advance, then accumulate phase at the new spacing — keeps frames coherent (the actual "vocoder" trick).
5. Inverse-FFT & overlap-add back to audio. **Pitch shift** then **resamples** to the original duration — pitch moves, length returns to normal.



$$X(f) = |X(f)|e^{j\angle X(f)} \quad \text{Robot: } Y(f) = |X(f)|e^{j0}$$

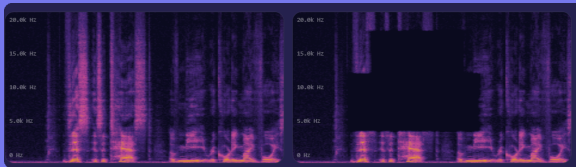
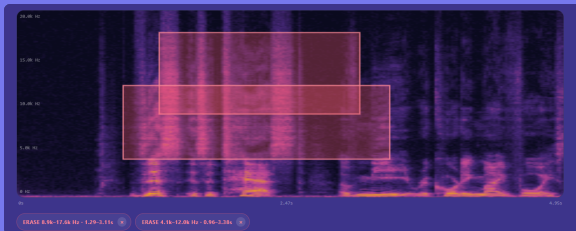
Robot skips all that: one STFT, phase forced to 0, inverse-STFT — magnitude survives, phase (voice's timing glue) does not. **Whisper** randomizes phase instead. Presets (Vader, Droid, Trooper, Cylon) layer pitch-shift/filter/ring-mod/reverb on top.

Spectral Portal — Spectrogram Masking

Algorithm

1. **Build the spectrogram:** windowed STFT frames give a time×frequency grid; colour encodes $|X(k, m)|$ in dB (purple = quiet, gold = loud).
2. **Paint KEEP/ERASE** rectangles → a binary mask $M(k, m)$, then Gaussian-blur it so edges are feathered, not a hard cut (avoids ringing).
3. **Multiply** the mask into the complex spectrum, bin by bin.
4. **Inverse STFT** (overlap-add) reconstructs the edited audio from what remains.

$$X'(k, m) = M(k, m) \cdot X(k, m)$$



Drawn erase regions (top) → masked spectrogram,
before/after (bottom)

Multi-Operation Signal Chain

Algorithm

1. Upload or record one input signal.
2. Add effect steps to a chain, in any order (up to 25).
3. Run the chain: each step's **output becomes the next step's input**.
4. Inspect per-step timing/levels, and compare runs in history.

$$y[n] = f_K(f_{K-1}(\cdots f_1(x[n]) \cdots))$$



Any of the 7 modules' effects can be a step — reorder, add, or remove freely.

CONCLUSION

Tech Stack & Summary

Backend Engine

Python • FastAPI • NumPy • SciPy •
Librosa • soundfile

Frontend Interaction

React • Vite • Modern CSS3 •
Canvas-based live visualizations

DSP Methods Covered

FFT & Spectrograms • Convolution • IIR/FIR
Filtering • Quantization & Aliasing • Phase
Vocoder • Spectral Masking

Thank You

Md Mostakim (2305063) • Samiul Basir Adry (2305071)